Universidad de La Sabana

Grafos

Representación y recorridos



Facultad de Ingeniería · Estructura de Datos y Algoritmos

Objetivos de la clase

1

Comprender

Qué es un grafo, sus tipos y qué problemas cotidianos modela.

2

Representar

Guardar un grafo en memoria (matriz o lista) y saber cuánto cuesta cada opción.

3

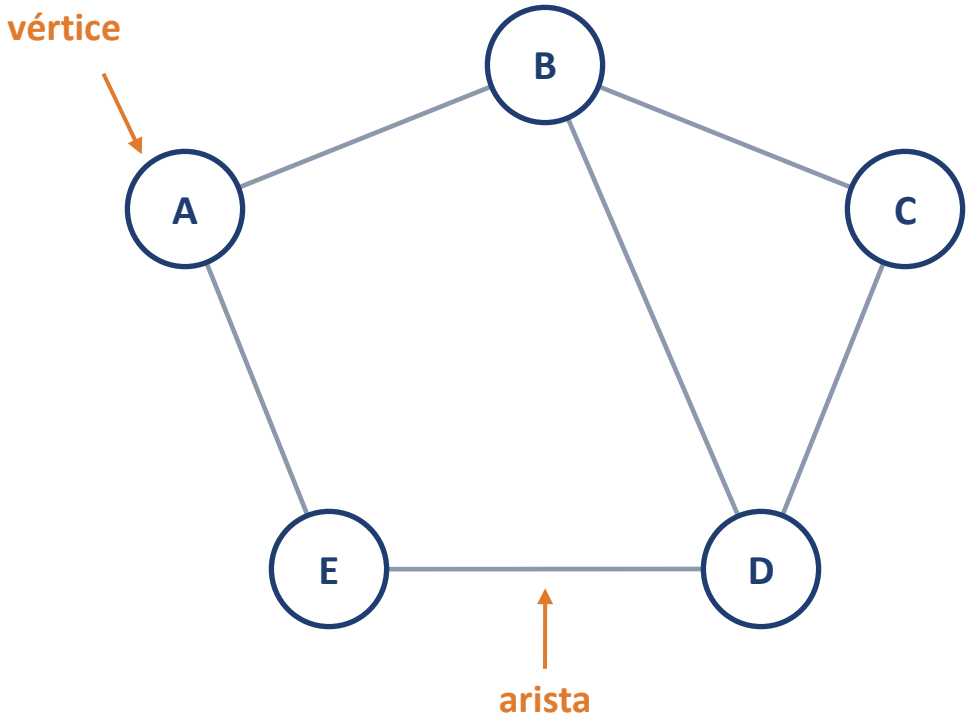
Recorrer

Aplicar BFS y DFS, y usarlos para resolver problemas: componentes, bipartición, caminos sin pesos.

Y ponerlo en práctica en la Tarea 3.

¿Qué es un grafo?

- Un grafo **G = (V, E)** es un conjunto de **vértices** V y un conjunto de **aristas** E que los conectan.
- Cada arista es una pareja de vértices: **no ordenada** {u, v} si el grafo es no dirigido, **ordenada** (u, v) si es dirigido.
- Notación del curso: **n = |V|** vértices y **m = |E|** aristas.



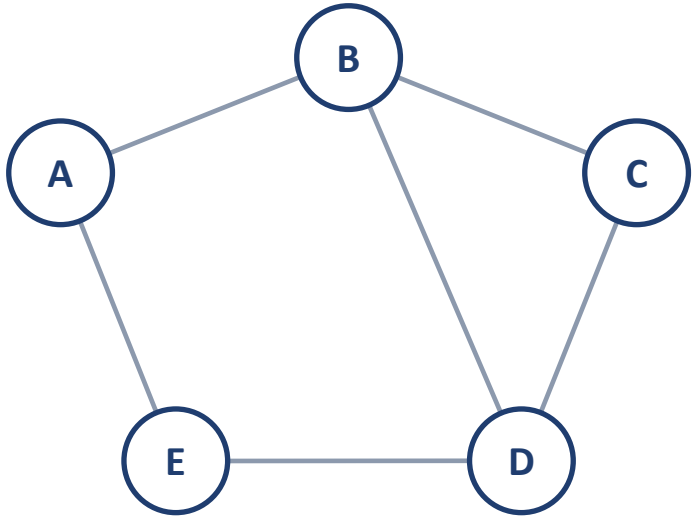
Tipos de grafos

El primer paso en cualquier problema de grafos es **determinar con qué tipo de grafo se está trabajando**.

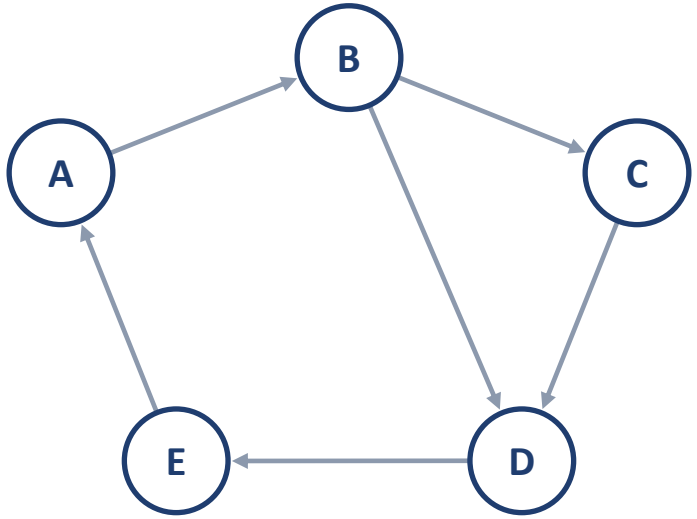
Dirigido	Ponderado	Simple	Disperso	Cíclico
vs	*vs*	*vs*	*vs*	*vs*
No dirigido	No ponderado	No simple	Denso	Acíclico

Dirigidos vs no dirigidos

- **No dirigido**: si la arista (x, y) está en E, también está (y, x).
- **Dirigido**: la existencia de (x, y) no implica la de (y, x).



No dirigido



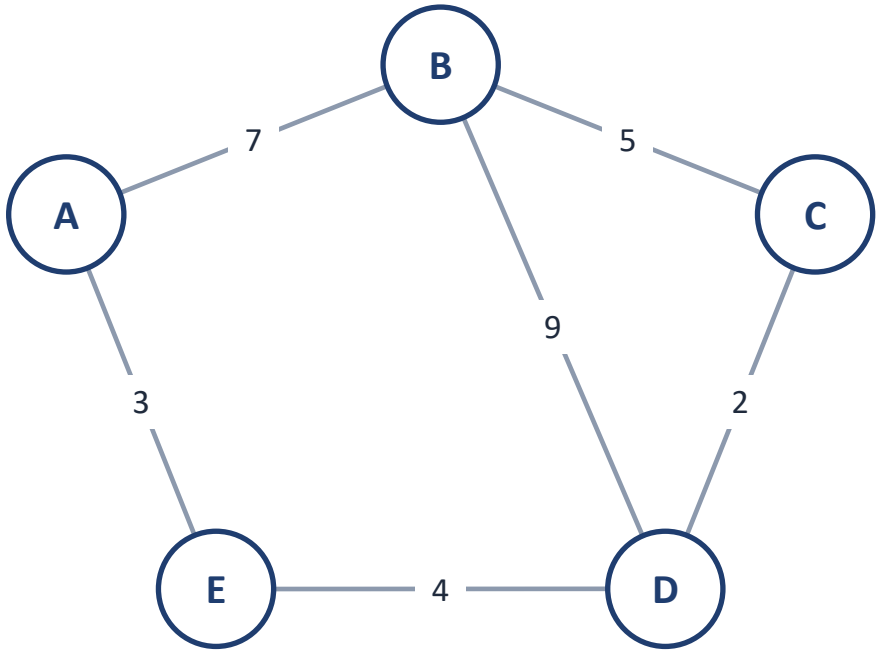
Dirigido

Piense

¿Las amistades de Facebook?
¿Los seguidores de Twitter / X?
¿Otros ejemplos?

Ponderados vs no ponderados

- **Ponderado**: cada arista tiene un **peso** w(u, v).
- **No ponderado**: todas las aristas pesan lo mismo; no hay distinción en el costo.

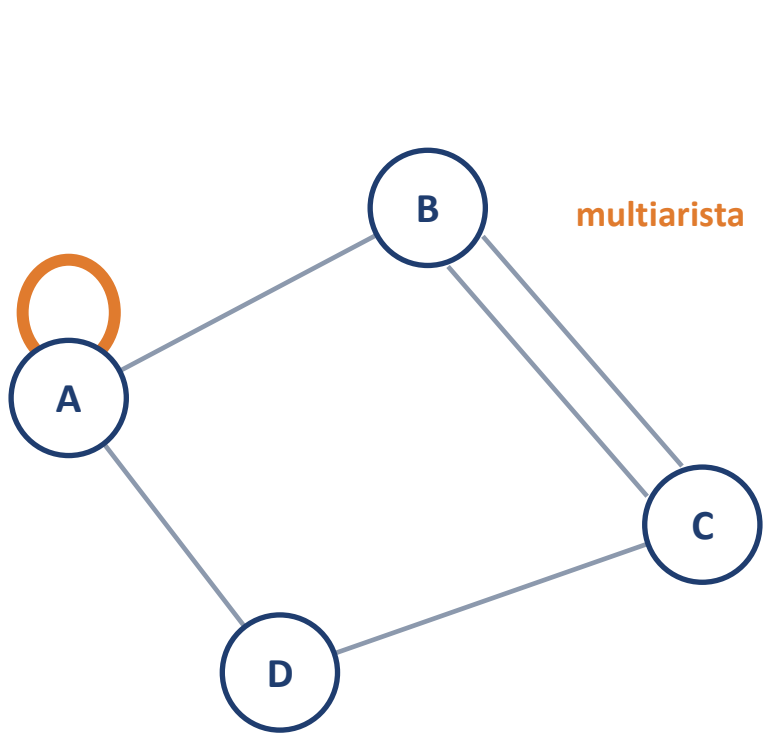


¿Qué pueden significar los pesos?

Piense en un mapa de carreteras, una red de computadores o un plan de proyecto.

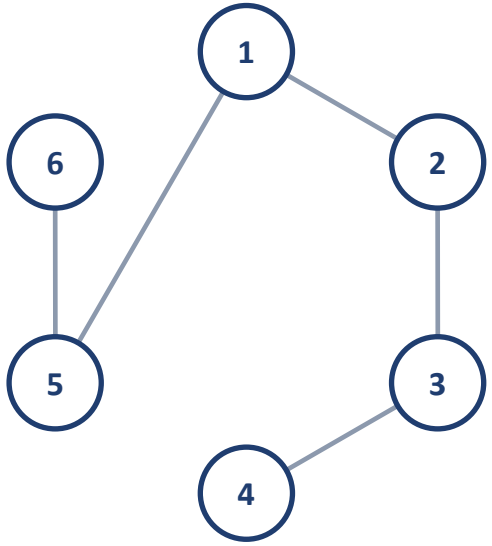
Simples vs no simples

- Ciertas aristas complican el trabajo con el grafo:
  - **Auto ciclo**: una arista (x, x) de un vértice a sí mismo.
  - **Multiarista**: la misma pareja (x, y) aparece más de una vez.
- Un grafo **simple** no tiene ninguna de las dos. Salvo que se diga lo contrario, en el curso los grafos son simples.

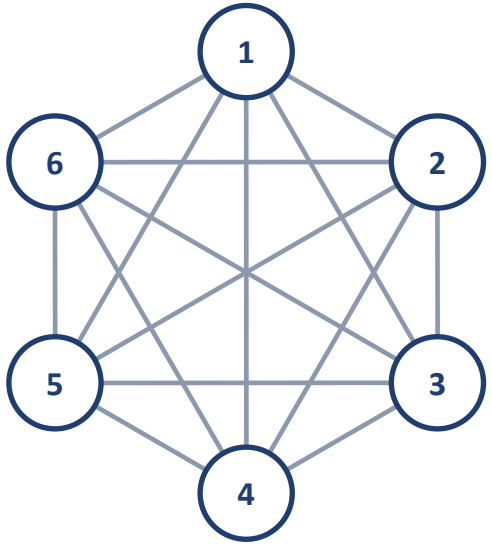


Dispersos vs densos

- **Disperso**: solo una pequeña fracción de los pares posibles tiene arista ($m \approx n$).
- **Denso**: la mayoría de los pares tiene arista ($m \approx n^2$).



Disperso: n = 6, m = 5



Denso: n = 6, m = 15

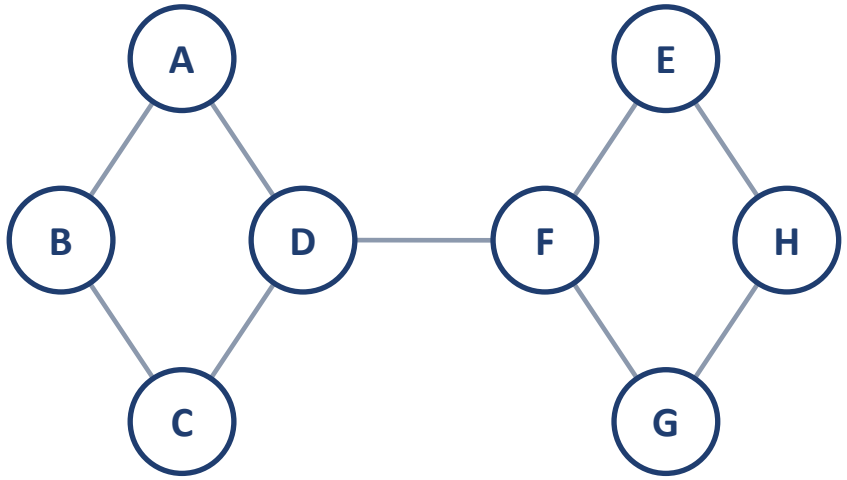
¿Cuántas aristas puede tener como máximo un grafo de n vértices?

R/ $n(n-1)/2$ (grafo simple no dirigido)

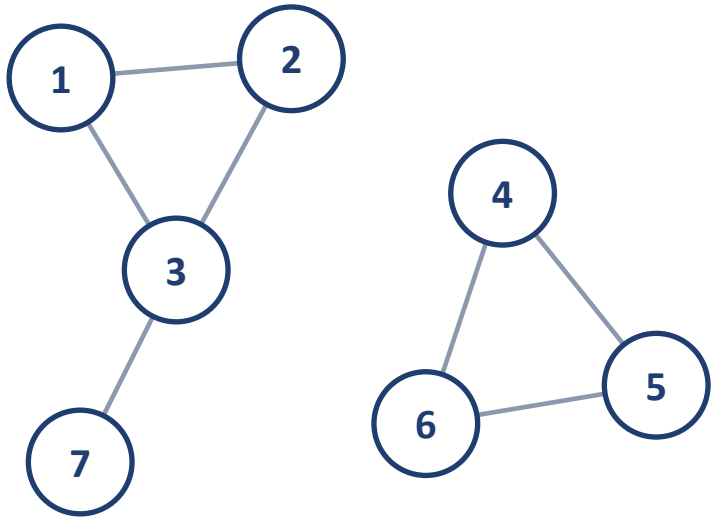
¿Facebook es un grafo denso o disperso?

¿Es un grafo conectado?

Un grafo es **conectado** si para cualquier par de vértices existe un **camino** que los une.



¿Es conectado?



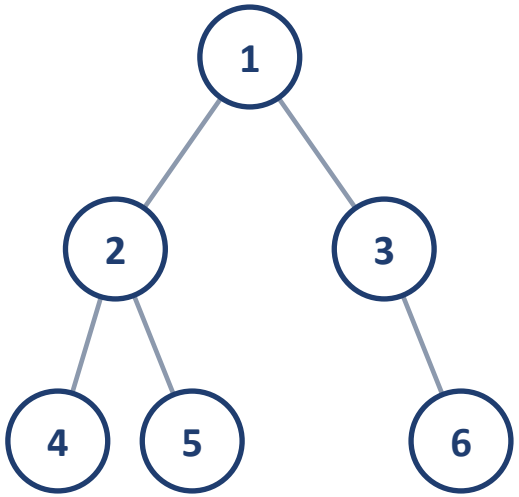
¿Y este?

Piense

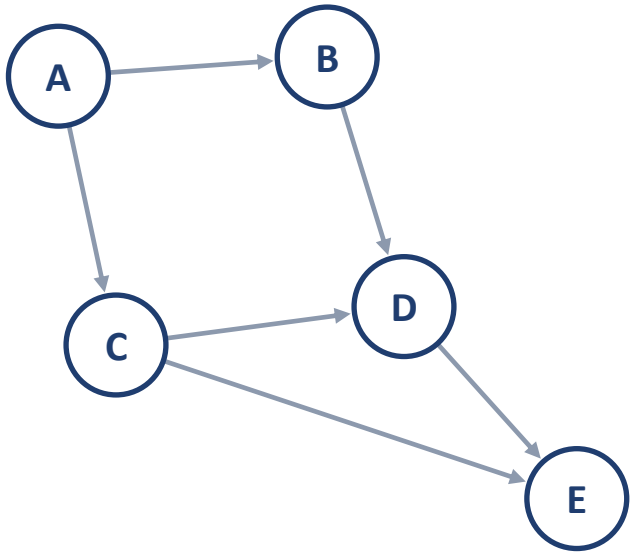
¿Cuál es el mínimo número de aristas para que un grafo de n vértices sea conectado?

Cíclicos vs acíclicos

- Un grafo **acíclico** no contiene ningún ciclo.
- Los **árboles** son grafos no dirigidos, conectados y acíclicos.
- ¿Y si es dirigido? Los **DAG** (directed acyclic graphs) son importantes en problemas de **planificación**.



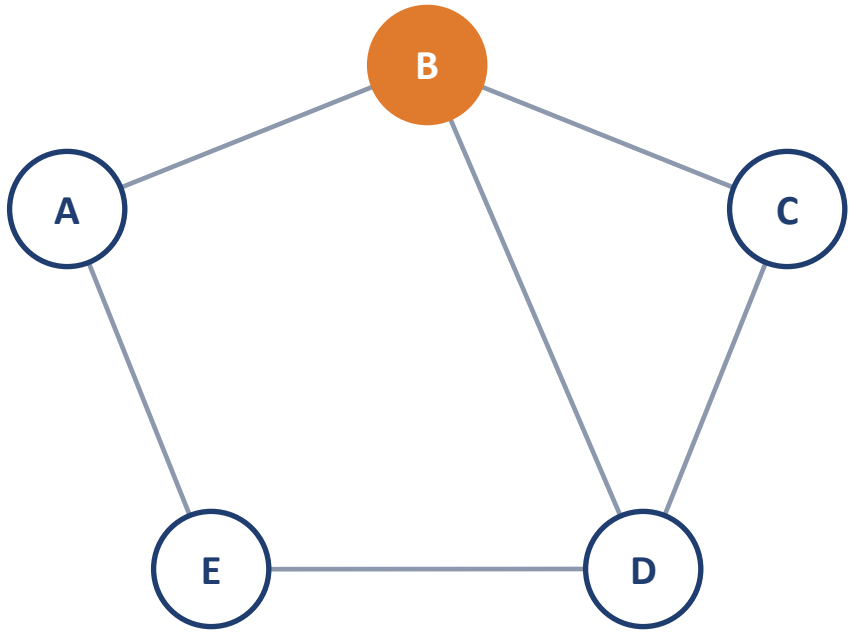
Árbol



DAG

Grado de un vértice

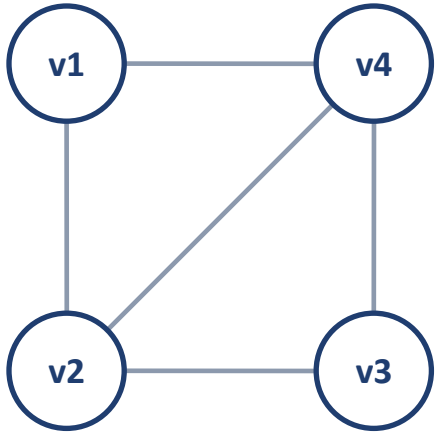
- El **grado** de un vértice es el número de aristas que inciden en él.
- En un grafo dirigido se distinguen el **grado de entrada** y el **grado de salida**.
- La suma de los grados de un grafo no dirigido es **2m**: cada arista se cuenta dos veces.



¿Cuál es el grado de B? ¿Y la suma de todos los grados?

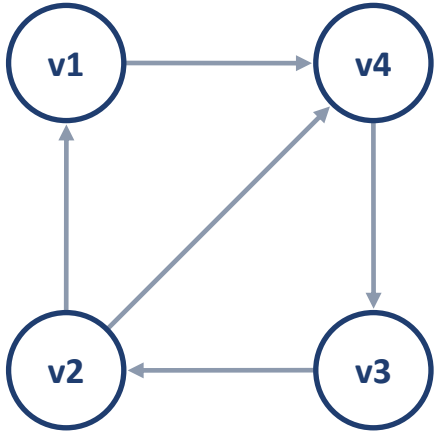
Matriz de adyacencia

- Para un grafo de n vértices, una matriz M de n × n: **M[i, j] = 1** si (i, j) es una arista y 0 si no.
- **Problema**: usa **O(n²)** de memoria, aunque el grafo tenga pocas aristas.
- Formas de ahorrar espacio:
  - Si el grafo es no dirigido, M es simétrica: basta la mitad.
  - Si el grafo es disperso: lista de adyacencia (siguiente).



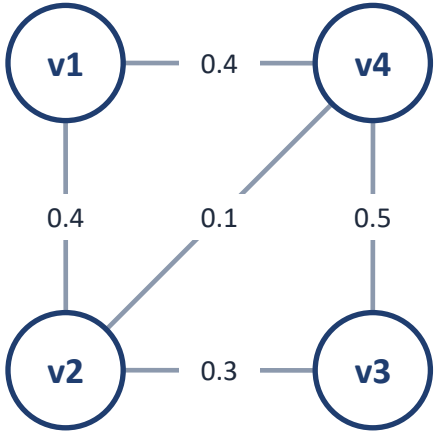
	v1	v2	v3	v4
v1	0	1	0	1
v2	1	0	1	1
v3	0	1	0	1
v4	1	1	1	0

Matriz de adyacencia: dirigido y ponderado



	v1	v2	v3	v4
v1	0	0	0	1
v2	1	0	0	1
v3	0	1	0	0
v4	0	0	1	0

Dirigido: M[i, j] = 1 si hay arista **de i a j**. La matriz deja de ser simétrica.

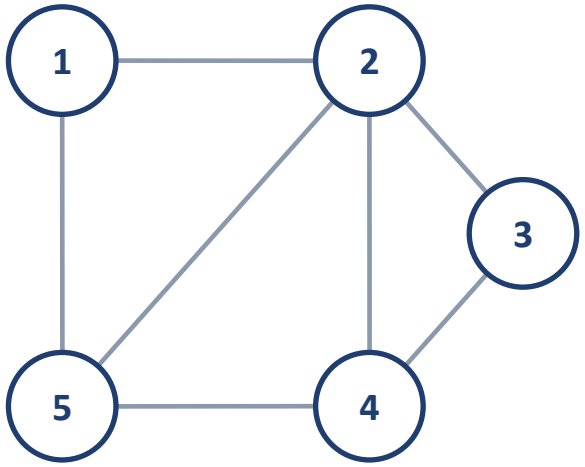


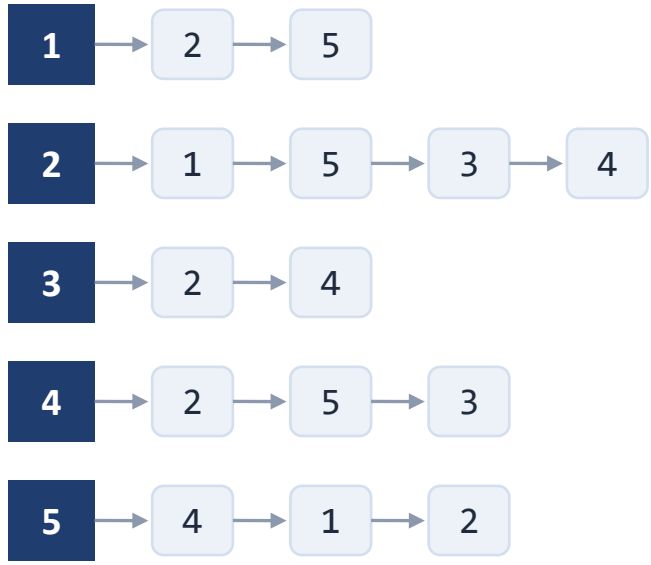
	v1	v2	v3	v4
v1	0	0.4	0	0.4
v2	0.4	0	0.3	0.1
v3	0	0.3	0	0.5
v4	0.4	0.1	0.5	0

Ponderado: M[i, j] guarda el **peso** de la arista (y un valor especial, como 0 o ∞, si no existe).

Lista de adyacencia

- Un arreglo de n listas: la lista i contiene los **vecinos** del vértice i.
- Memoria **O(n + m)**: en un grafo no dirigido hay 2m entradas, porque cada arista aparece en las dos listas.
- Saber si (i, j) es arista cuesta **O(d_i)**, donde d_i es el grado de i.





Memoria

O(n + m)

Matriz vs lista: ¿cuál gana?

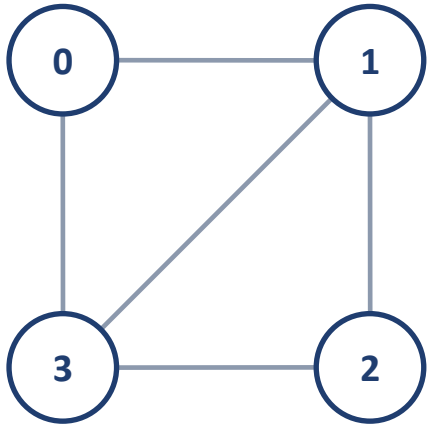
Pregunta	Ganador (matriz / lista)
¿Más rápido para probar si la arista (i, j) existe?	
¿Más rápido para hallar el grado de un vértice?	
¿Menos memoria en un grafo disperso?	
¿Más rápido para insertar o eliminar una arista?	
¿Más rápido para recorrer todo el grafo?	

¿Qué representación cree que usa Facebook para su red de amistades?

Un grafo en GCL

- GCL solo tiene arreglos. La lista de adyacencia se guarda en dos:
  - `grado[u]`: cuántos vecinos tiene u.
  - `ady[u,i]`: el i-ésimo vecino de u, con 0 ≤ i < grado[u].
  - `c[u,i]`: costo de la arista (u, ady[u,i]), si hay pesos.
- Los vértices son 0, 1, …, n − 1.
- Recorrer los vecinos de u cuesta O(grado[u]): un recorrido completo cuesta **O(n + m)**.

```
con n : nat                              {vértices 0, …, n-1}
con grado : array[0,n) of nat
con ady : array[0,n)x[0,n) of nat
con c : array[0,n)x[0,n) of int          {solo si hay pesos}
```



u	grado[u]	ady[u,0]	ady[u,1]	ady[u,2]
0	2	1	3	
1	3	0	2	3
2	2	1	3	
3	3	0	1	2

Segunda parte

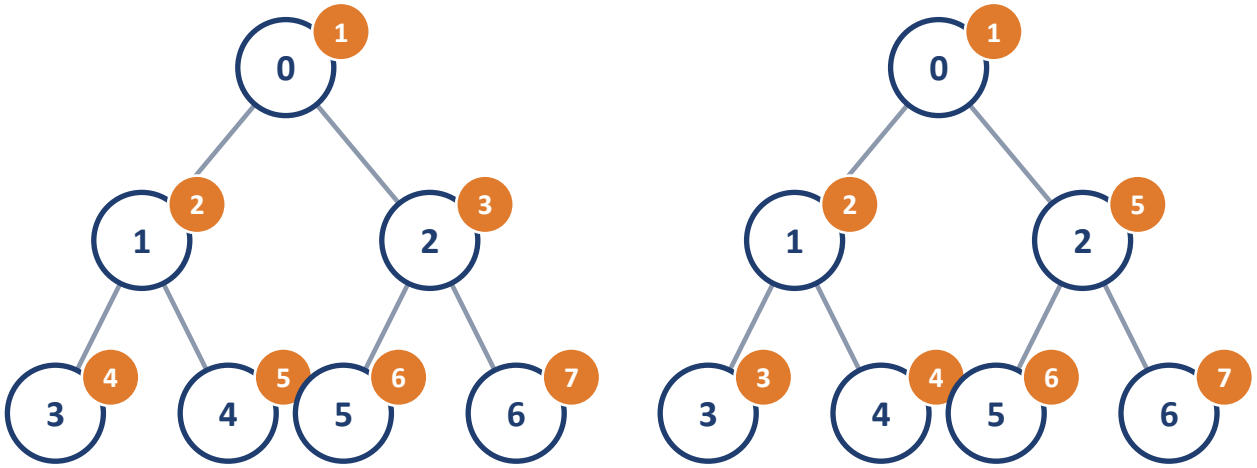
Recorridos: BFS y DFS



Universidad de La Sabana

Recorridos de un grafo

- Recorrer es visitar **todos los vértices alcanzables** desde un vértice inicial s, cada uno **una sola vez**.
- **BFS** (en anchura): visita por niveles; usa una **cola**.
- **DFS** (en profundidad): avanza lo más lejos posible y retrocede; usa una **pila** o recursión.
- Son la base de: componentes conexas, bipartición, caminos más cortos sin pesos, orden topológico.



BFS: por niveles **DFS: en profundidad**

Los números naranja indican el orden de visita.

Ambos, con lista de adyacencia

$O(n + m)$

BFS: búsqueda en anchura

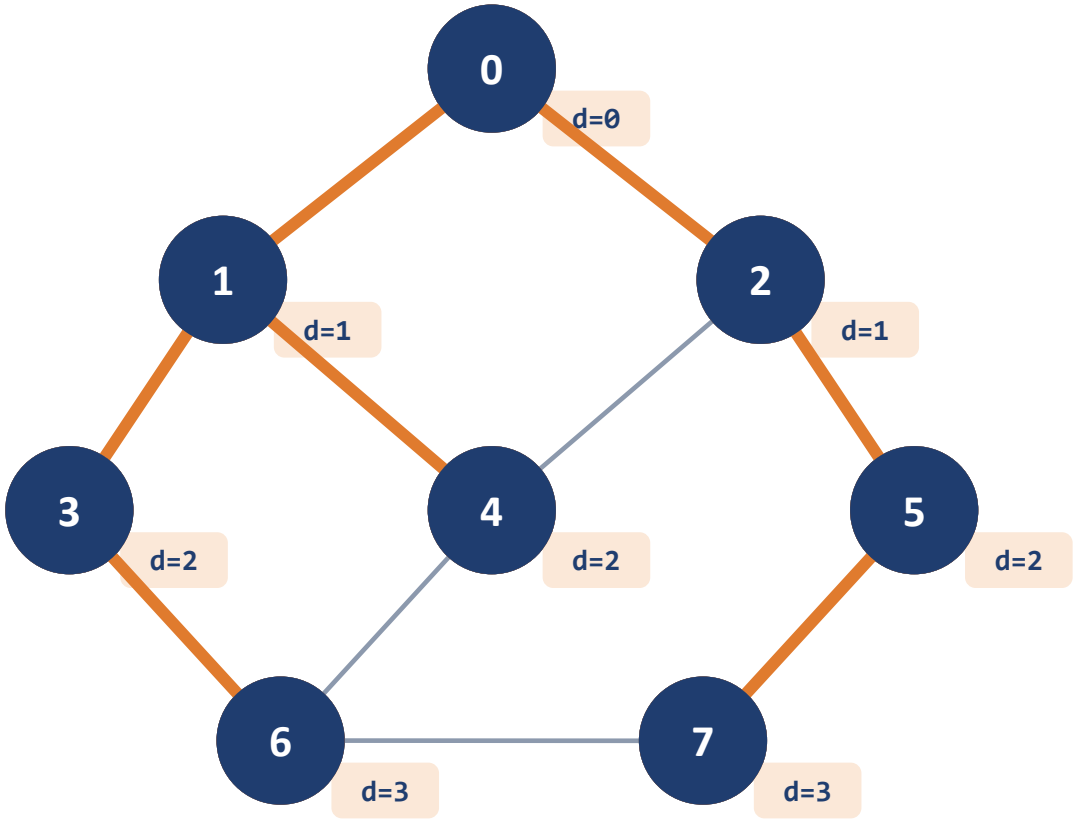
```
con s : nat
var d : array[0,n) of int          {-1: no descubierto}
var Q : array[0,n) of nat          {la cola es Q[ini..fin)}
var ini, fin, u, i, v : nat
{Pre: 0 ≤ s < n}
v := 0;
do v < n → d[v] := -1; v := v + 1 od;
d[s], Q[0], ini, fin := 0, s, 0, 1;
do ini < fin →
   u, ini, i := Q[ini], ini + 1, 0;
   do i < grado[u] →
      v := ady[u,i];
      if d[v] = -1 → d[v], Q[fin], fin := d[u] + 1, v, fin + 1
      [] d[v] ≠ -1 → skip
      fi;
      i := i + 1
   od
od
{Post: d[v] = mín. aristas de s a v; -1 si no se alcanza}
```

- d[v] = **mínimo número de aristas** de s a v: camino más corto en un grafo **no ponderado**.
- La cola es Q[ini..fin): se saca por ini y se mete por fin. Cada vértice entra una sola vez, así que basta un arreglo de tamaño n.

Complejidad

$O(n + m)$

BFS en acción



Naranja: actual · Azul claro: en la cola · Azul: terminado

Cola vacía: fin. Las aristas naranja forman el árbol BFS; d[v] es la distancia mínima en aristas.

Cola (frente a la izquierda)

```
vacía
```

Orden de visita

```
0, 1, 2, 3, 4, 5, 6, 7
```

DFS: búsqueda en profundidad

```
var visto : array[0,n) of bool

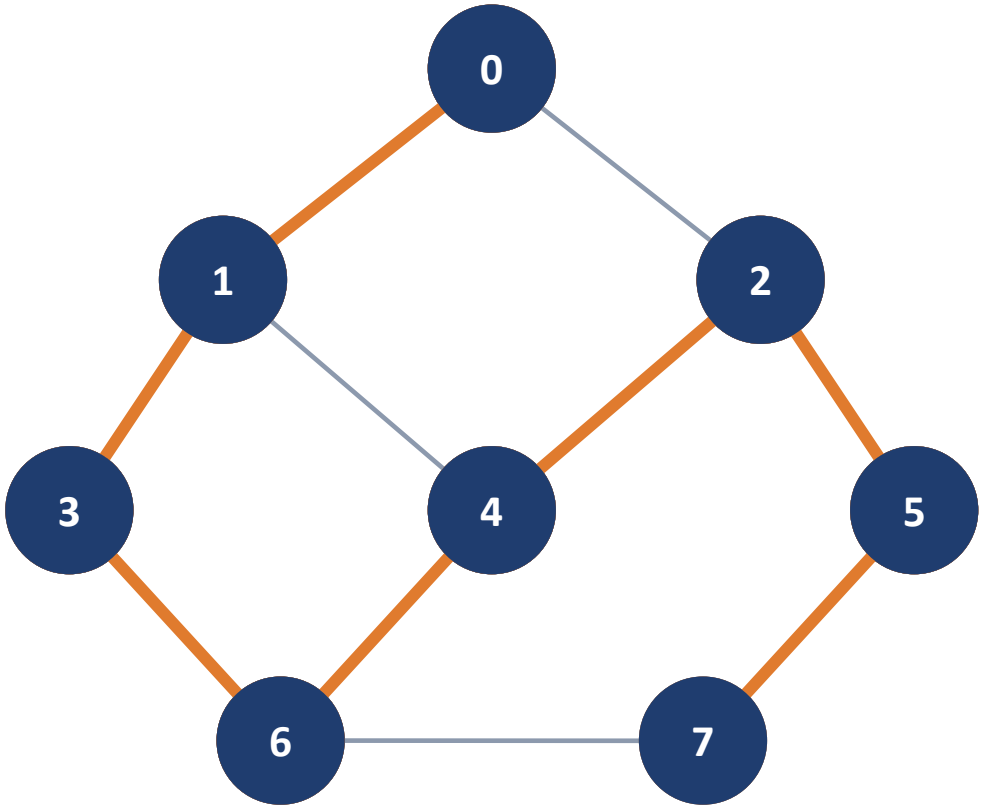
proc dfs ( u : nat )
   {Pre: ¬visto[u]}
   var i : nat
   visto[u], i := true, 0;
   do i < grado[u] →
      if ¬visto[ady[u,i]] → dfs(ady[u,i])
      [] visto[ady[u,i]] → skip
      fi;
      i := i + 1
   od
```

- La recursión hace de **pila**: avanza por un vecino no visitado y retrocede cuando no quedan.
- Cada llamada tiene su propia variable i: al volver, sigue con el siguiente vecino.
- Para recorrer todo el grafo: llamar dfs desde cada vértice todavía no visitado.

Complejidad

$O(n + m)$

DFS en acción



Naranja: actual · Azul claro: en la pila · Azul: terminado

0 termina: fin del recorrido. Las aristas naranja forman el árbol DFS.

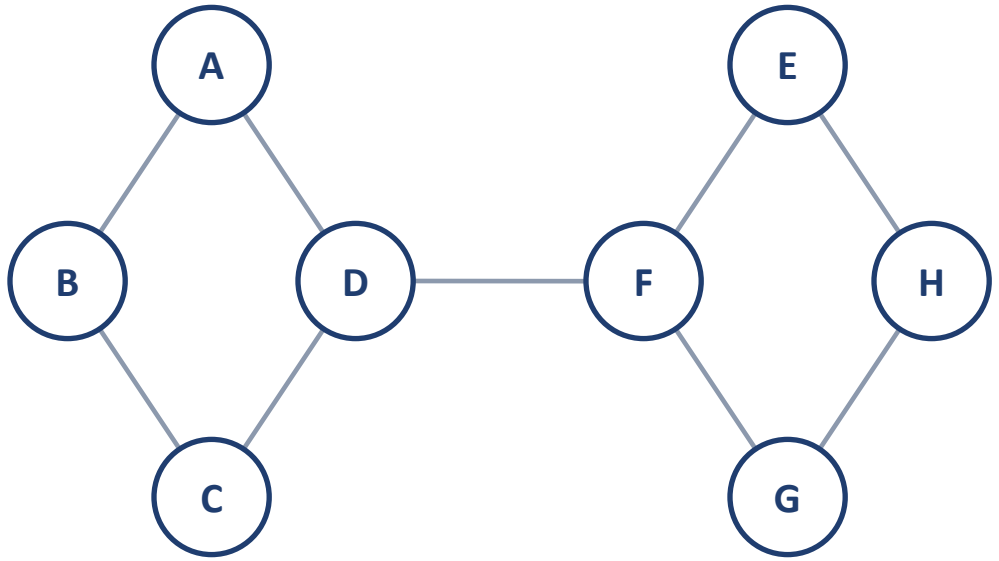
Pila de recursión (tope a la derecha)

`vacía`

Orden de visita

`0, 1, 3, 6, 4, 2, 5, 7`

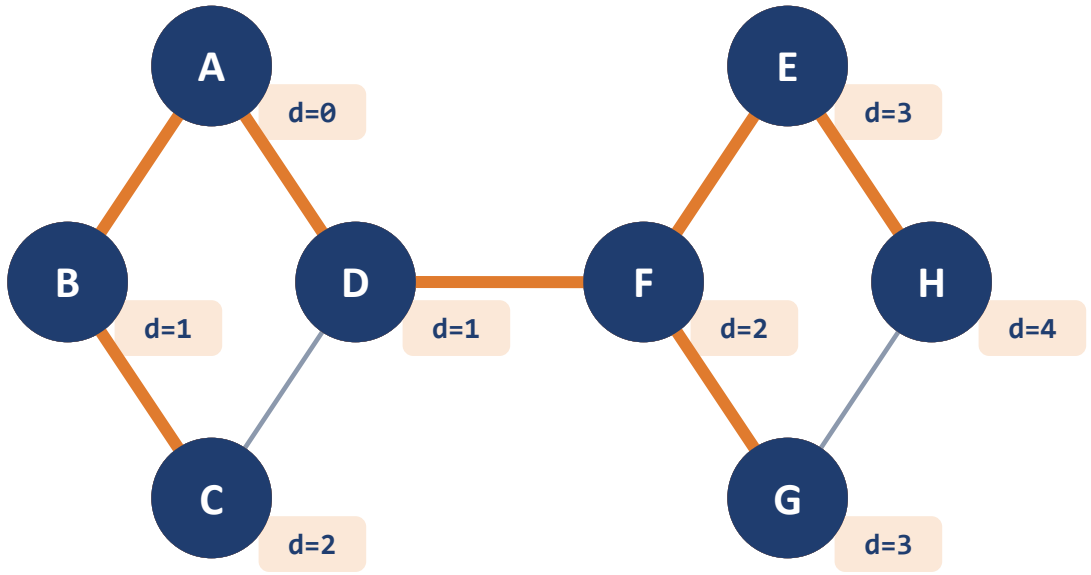
Ejercicio



Recorra el grafo desde **A**, tomando los vecinos en **orden alfabético**:

1. ¿En qué orden visita los vértices BFS? ¿Cuánto vale d[v] para cada uno?
2. ¿En qué orden los visita DFS?
3. ¿Cuántas componentes quedan si se elimina la arista D–F?
4. ¿Se puede pintar con dos colores sin que una arista una vértices del mismo color?

Solución: BFS desde A



Cola vacía: fin. Las aristas naranja forman el árbol BFS; d[v] es la distancia mínima en aristas.

Cola (frente a la izquierda)

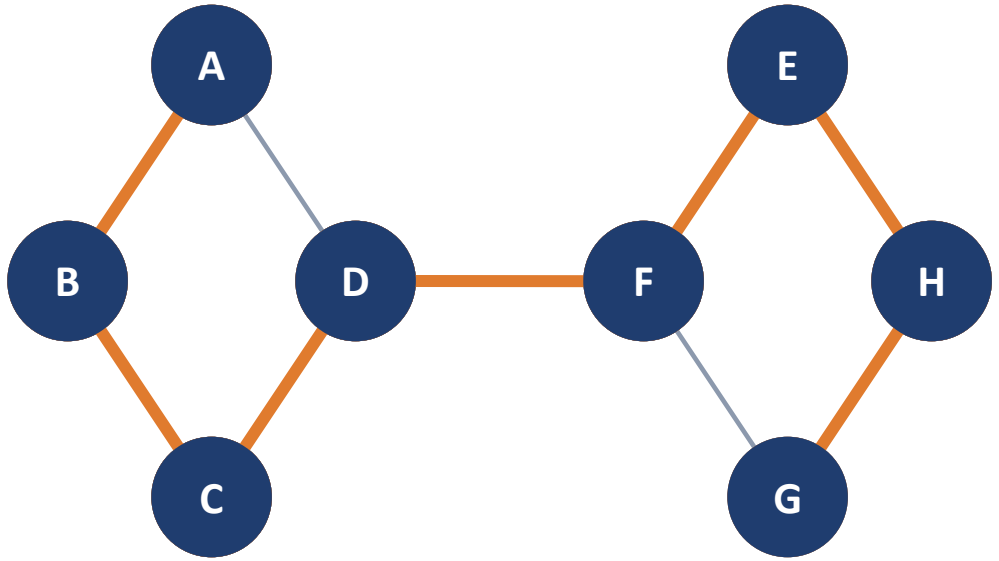
vacía

Orden de visita

A, B, D, C, F, E, G, H

Naranja: actual · Azul claro: en la cola · Azul: terminado

Solución: DFS desde A



A termina: fin del recorrido. Las aristas naranja forman el árbol DFS.

Pila de recursión (tope a la derecha)

`vacía`

Orden de visita

A, B, C, D, F, E, H, G

Naranja: actual · Azul claro: en la pila · Azul: terminado

Aplicación: componentes conexas

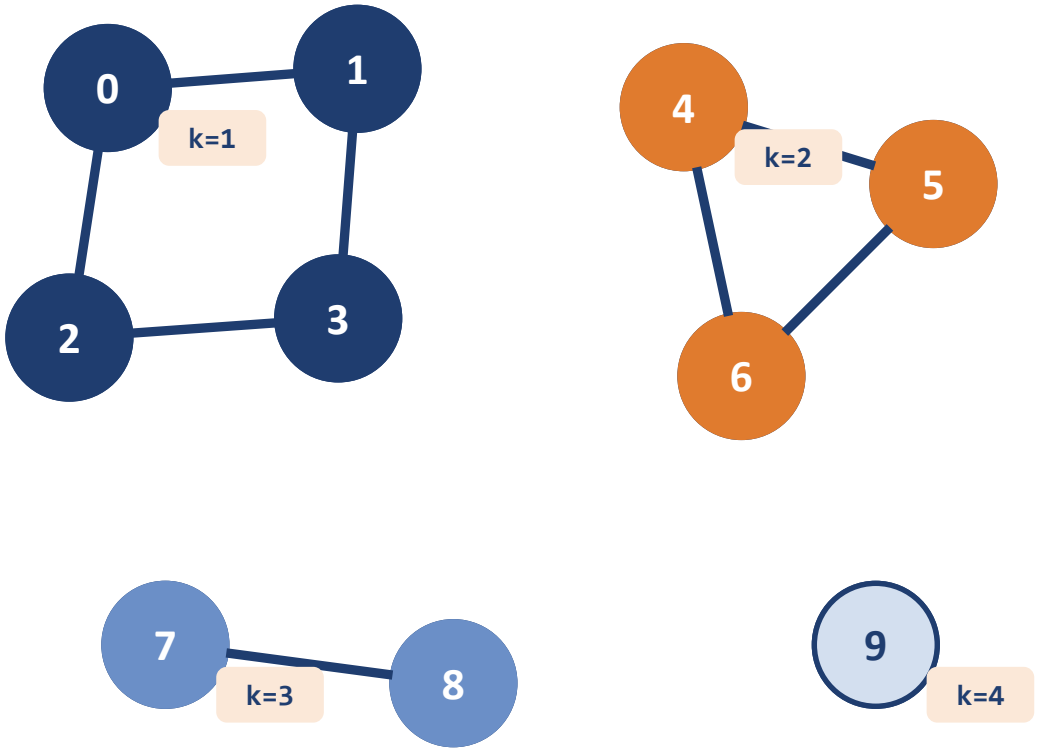
```
var k, v : nat
v := 0;
do v < n → visto[v] := false; v := v + 1 od;
k, v := 0, 0;
do v < n →
   if ¬visto[v] → k := k + 1; dfs(v)
   [] visto[v] → skip
   fi;
   v := v + 1
od
{Post: k = número de componentes conexas}
```

- Componente conexa: conjunto **máximo** de vértices conectados entre sí.
- Cada recorrido que arranca en un vértice no visitado descubre una componente completa.
- El grafo es conectado ⇔ **k = 1**.

En total, todos los recorridos

O(n + m)

Componentes conexas en acción

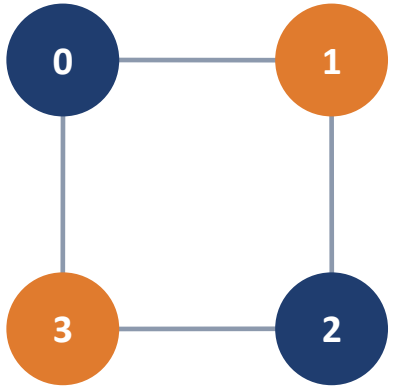


Los demás vértices ya estaban visitados. Resultado: k = 4 componentes conexas.

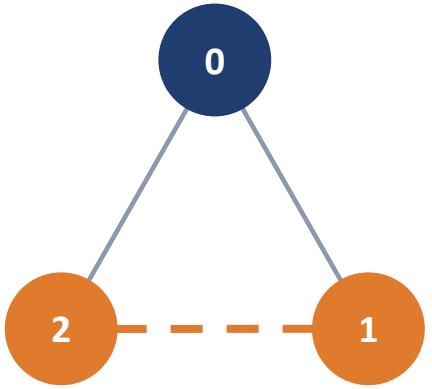
k = 4

Aplicación: ¿el grafo es bipartito?

- **Bipartito**: los vértices se pintan con 2 colores sin que una arista una dos del mismo color.
- Es bipartito ⇔ no tiene **ciclos de longitud impar**.
- Con BFS: cada vecino recibe el color contrario. Si una arista une dos del mismo color, no es bipartito.
- **Si el grafo no es conexo, se repite desde cada vértice sin color**: el ciclo impar puede estar en cualquier componente.



Ciclo de 4: bipartito



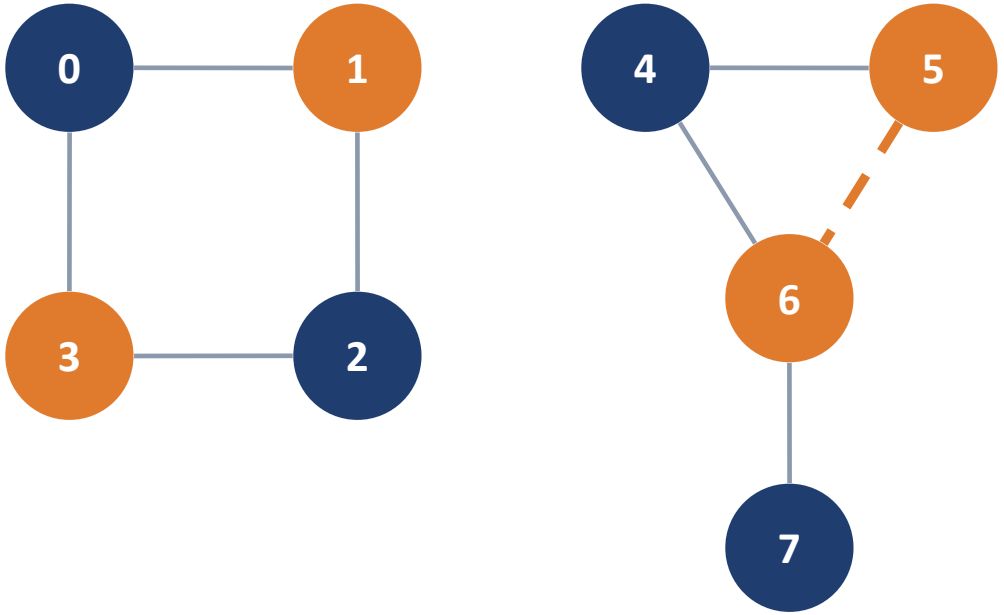
Ciclo de 3: no lo es

Bipartito en GCL

```
var color : array[0,n) of int   {-1: sin color}
var Q : array[0,n) of nat
var ok : bool
var s : nat
s := 0;
do s < n → color[s] := -1; s := s + 1 od;
ok, s := true, 0;
do s < n →                          {cada componente}
   if color[s] = -1 → colorear(s)
   [] color[s] ≠ -1 → skip
   fi;
   s := s + 1
od
{Post: ok ⇔ el grafo es bipartito}
```

```
proc colorear ( s : nat )       {BFS desde s}
   var ini, fin, u, i, v : nat
   color[s], Q[0], ini, fin := 0, s, 0, 1;
   do ini < fin →
      u, ini, i := Q[ini], ini + 1, 0;
      do i < grado[u] →
         v := ady[u,i];
         if color[v] = -1 → color[v] := 1 - color[u];
                            Q[fin], fin := v, fin + 1
         [] color[v] = color[u] → ok := false
         [] color[v] = 1 - color[u] → skip
         fi;
         i := i + 1
      od
   od
```

Bipartito en acción



Fin: ok = false. El ciclo impar está en la segunda componente: un solo BFS desde 0 no lo habría encontrado.

```
ok = false
```

Azul: color 0 · Naranja: color 1 · Anillo: vértice que sale de la cola · Línea punteada: conflicto

BFS vs DFS

	BFS	DFS
Estructura	Cola (FIFO)	Pila (LIFO) o recursión
Orden de visita	Por niveles: primero los más cercanos a s	Lo más profundo posible; luego retrocede
Camino más corto sin pesos	Sí: d[v] es la distancia mínima	No
Complejidad	O(n + m)	O(n + m)
Aplicaciones típicas	Distancias sin pesos, bipartición	Componentes, detección de ciclos, orden topológico

Orden topológico (en un DAG)

Ordenar los vértices de modo que toda arista u → v deje u antes que v. Con DFS: anotar cada vértice cuando termina de explorarse y luego invertir la lista.



¿Preguntas?